INTEGRAL DRAGON

MODULE 02 · SESSION 01

What Good Looks Like

Define project health in practical terms.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

Why this course starts with "what good looks like"

- **You need a quality target first.** If you cannot define healthy project qualities, AI will optimize the wrong thing.
- **More code is not automatically progress.** AI amplifies both clarity and confusion.
- **A good project is easier to change.** Understandable, testable, debuggable, observable, operable.
- **Debt comes next.** You cannot diagnose friction without a picture of what "healthy" looks like.

A healthy project has more than clean code

Clean code is one slice

Style, naming, and internal hygiene matter — but they are a small fraction of how changeable a system really is.

- readable modules
- reasonable names
- low duplication

Project health is broader

Docs, environment setup, release process, and operational visibility are all part of software quality.

- docs you can trust
- setup that reproduces
- runtime you can see
- releases you can verify

Teams suffer when these are weak even if the code style looks decent.

Nine categories of project health

CATEGORY 01

Documentation & discoverability

A repo that tells the next engineer how to understand, run, and change it.

CATEGORY 02

Architecture clarity & boundaries

Clear responsibilities, dependency direction, useful diagrams.

CATEGORY 03

Reproducible developer setup

Clean-machine onboarding with bounded effort and no oral tradition.

CATEGORY 04

Testability & quality confidence

Tests that help you move, not vanity coverage. Confidence, not theater.

CATEGORY 05

Build, deploy & release discipline

Repeatable builds, clear versions, verifiable deployments.

CATEGORY 06

Data, schema & environment

Reviewable migrations, trustworthy seed data, reversible changes.

CATEGORY 07

Observability & operability

Logs, traces, metrics, and deploy visibility that support diagnosis.

CATEGORY 08

Security & supply chain

Secrets managed, dependencies scanned, secure defaults deliberate.

CATEGORY 09

Workflow & collaboration

Small reviewable changes, clear ownership, AI output reviewed like any diff.

The top 12 "what good looks like" checks

01 **README is trustworthy.**

02 **Architecture is understandable** at a glance.

03 **Local setup is reproducible.**

04 **Project builds** in a clean environment.

05 **Tests provide real confidence.**

06 **Schema changes** are versioned and reviewable.

07 **Test data is intentional,** not accidental.

08 **Versions and releases are visible.**

09 **Deployments can be verified.**

10 **Logs, metrics, and traces** explain behavior.

11 **Secrets and dependency risk** are managed deliberately.

12 **Workflow supports** small, repeatable improvements.

Four nuances to teach on day one

TESTABILITY

Not "run the full system locally"

For distributed systems, the better standard is **meaningful local or isolated testability** — not perfect local parity.

CONFIDENCE

Not "high coverage"

Coverage is a count. Confidence is a capability. Teach **risk-relevant testing**, not number-chasing.

SECURITY

Not "no passwords in code"

Teach **secrets management and secure default handling** — including dependency and supply-chain hygiene.

OBSERVABILITY

Not "just logs"

Teach **logs + traces + metrics + deploy visibility + debugging usefulness**. It is part of maintainability.

Exercise: Repository Health Scorecard

What you will do

- Review a real repo: README, structure, workflows, visible docs.
- Score each of the nine categories from 1 to 5.
- Write one sentence explaining each low score.
- Identify the three weaknesses most likely to slow future change.

How to use AI

- Ask AI to summarize repo structure and surface missing docs.
- Ask AI to list visible testing and CI signals.
- Do not let AI assign scores — humans own the judgment.
- Ground every score in actual repo evidence.

Output: a scored rubric, a top-three problem list, and a one-paragraph conclusion. Time: 15–20 minutes in pairs or small groups.

• CLOSE OUT

The point is not more code. The point is easier change.

Good engineering fundamentals do not compete with AI. They make AI more powerful. Tomorrow: the friction and debt that block "good" — and how to see it clearly.